A2 — Sigma-delta 1 bit

TP

TP avancé (optionnel) — Signal v1

Guilhem PAGES **Version 1**
guilhem.pages@univ-lemans.fr **Durée : 2 h – 3 h**

? Problématique

Combien de bits faut-il vraiment pour coder de l'audio ? Et comment les convertisseurs « 24 bits » du commerce obtiennent-ils cette précision avec un comparateur **1 bit** ? On répond en trois étapes : requantifier un signal (bruit, dither, mise en forme), simuler un **CAN sigma-delta** complet, puis fabriquer un **CNA 1 bit** sur une simple broche de la Teensy.

Objectifs:

- Mesurer le bruit de quantification (SINAD, SNR, THD, ENOB) et le comparer à la formule $6{,}02\,B + 1{,}76$ dB ;
- Comprendre le rôle du **dither** et de la **mise en forme du bruit** ;
- Programmer un modulateur **sigma-delta** d'ordre 1 et 2 et un filtre de décimation **CIC** ;
- Relier performances, ordre et taux de suréchantillonnage (OSR) ;
- Émettre un flux 1 bit à 11,3 MHz sur une broche et le filtrer en analogique.

Prérequis:

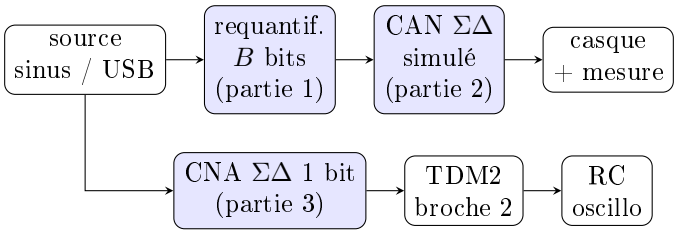
- Projet compresseur–limiteur, partie C (suréchantillonnage) conseillée ;
- Quantification, puissance d'un signal, décibels ;
- Transformée en z, fonction de transfert d'un intégrateur numérique.

® Remarque : Matériel et réglages

Teensy 4 + Audio Shield, casque ; partie 3 : 2 résistances (1 kΩ, 10 kΩ), 2 condensateurs (4,7 nF, 470 pF), oscilloscope (avec FFT si possible). *Outils* → *USB Type* → *« Serial + MIDI + Audio »*, moniteur série à 115 200 bauds, pupitre preset **A2 – Quantification** / ΣΔ.

® **Remarque** : Pilotage en MIDI

CC	Paramètre	Valeurs	Loi
21	niveau de la source	$-60 - 0$ dBFS	linéaire (en dB)
24	fréquence du sinus de test	$200 - 10\,000$ Hz	logarithmique
25	nombre de bits B	2, 3, 4, 6, 8, 12, 16	sélecteur
26	mise en forme du bruit	ordre 0, 1, 2	sélecteur
27	CAN sigma-delta	arrêt, ordre 1, ordre 2	sélecteur
28	OSR	16, 32, 64	sélecteur
29	bande de mesure du SNR	$1 - 20$ kHz	logarithmique
81	source	sinus de test / USB	interrupteur
82	dither TPDF	non / oui	interrupteur
83	flux 1 bit sur la broche 2	arrêt / marche	interrupteur



1 Le banc de mesure (fourni)

`mesure.h/.cpp` capture $N = 4096$ échantillons de sortie et calcule leur FFT. Le sinus de test est **cohérent** : $f_0 = m\,F_s/N$ avec m entier impair, si bien qu'il fait exactement m périodes dans la fenêtre. Sans aucune fenêtre de pondération, chaque case de la FFT se lit directement : case m = signal, cases hm = harmoniques, le reste = bruit. Le moniteur série affiche :

SINAD	signal / (bruit + distorsion), toute la bande $0 - F_s/2$
SNR	signal / bruit (harmoniques 2 à 10 exclues)
THD	harmoniques 2 à 10 / signal (dBc)
SNR[0–f]	signal / bruit dans la bande $[0, f]$ (CC 29)
ENOB	nombre effectif de bits : $(\text{SINAD} - 1{,}76 - \text{niveau}_{\text{dBFS}})/6{,}02$

Activité 1

Q1. Téléverser le squelette `avance/A2_sigma_delta/etudiant/` (tout est en « arrêt », $B = 16$). Relever SINAD et ENOB pour un sinus à -6 dBFS. D'où vient cette limite ? C'est le **plancher** du banc de mesure.

2 Partie 1 – Quantification, dither, mise en forme du bruit

Quantification. On ramène le signal normalisé $x \in [-1, 1[$ sur 2^B niveaux de pas $q = 2/2^B$: $y = q \cdot \text{arrondi}(x/q)$. Si le signal est « assez riche », l'erreur $e = y - x$ est uniforme sur $[-q/2, q/2]$, de

puissance $q^2/12$. Pour un sinus pleine échelle (puissance 1/2) :

$$\mathrm{SNR} = 10\log\frac{1/2}{q^2/12} = 6{,}02\,B + 1{,}76\ \mathrm{dB} \qquad \text{(moins |niveau| si le sinus est plus faible).}$$

Activité 2

Q2. Dans `effect_requantizer.cpp`, écrire la quantification « mid-tread » (`yq`) à partir de `v`.
Q3. Sinus à -6 dBFS : mesurer SINAD et ENOB pour $B = 4$ et $B = 8$ et comparer à la formule. Écouter ensuite de la musique (source USB) à $B = 8, 6, 4$.

```
// TODO (Q2) : quantification « mid-tread » au pas q :
//             yq = q · arrondi(v / q)   (arrondi(u) = floorf(u + 0.5f))
// TODO (Q4) : ajouter avant l'arrondi un dither TPDF d = q·(aleatoire() - aleatoire())
//             si dith est vrai
// TODO (Q6) : mise en forme du bruit -- partir de v = x - (retour d'erreur) :
//             ordre 1 : v = x - e1          ordre 2 : v = x - 2·e1 + e2
//             puis, après quantification : e = yq - v ; e2 = e1 ; e1 = e
float v  = x;                   // provisoire
float yq = v;                   // provisoire : aucune quantification
```

Dither. Pour un signal faible (quelques pas q), l'erreur devient une fonction déterministe du signal : c'est de la *distorsion*, très audible sur les fins de notes. On ajoute avant l'arrondi un bruit **TPDF** (densité triangulaire, somme de deux uniformes, amplitude $\pm q$) qui rend l'erreur indépendante du signal, au prix d'un peu plus de bruit (puissance totale $q^2/4$ au lieu de $q^2/12$).

Activité 3

Q4. Ajouter le dither : `d = q * (aleatoire() - aleatoire())` si `dith` est vrai, à ajouter à `v` avant l'arrondi.
Q5. $B = 8$, sinus à -40 dBFS : relever SINAD et THD sans puis avec dither. Recommencer à -6 dBFS. Conclure. Écouter un morceau calme, $B = 6$, avec et sans dither.

Mise en forme du bruit (*noise shaping*). On réinjecte l'erreur commise aux échantillons précédents :

$$v[n] = x[n] - \big(2e[n-1] - e[n-2]\big), \qquad y[n] = Q\big(v[n]\big), \qquad e[n] = y[n] - v[n]$$
$$\Longrightarrow \quad Y(z) = X(z) + (1 - z^{-1})^2\,E(z).$$

L'erreur est filtrée par $\mathrm{NTF}(z) = (1-z^{-1})^L$ (ordre $L = 1$: $v = x - e[n-1]$), de module $|2\sin(\pi f/F_s)|^L$: nulle en basse fréquence, maximale à $F_s/2$.

Activité 4

Q6. Écrire le retour d'erreur (ordres 0, 1, 2) et la mise à jour de `e1`, `e2`. L'erreur est calculée *avant* la saturation : pourquoi ?
Q7. $B = 8$, sinus à -6 dBFS, bande de mesure 4 kHz : remplir le tableau (sans dither). À quelle fréquence la mise en forme commence-t-elle à *augmenter* le bruit ? Écouter de la musique à $B = 6$ avec l'ordre 2.

ordre	SINAD $(0 - F_s/2)$	SNR[0 – 4 kHz]	écoute
0			
1			
2			

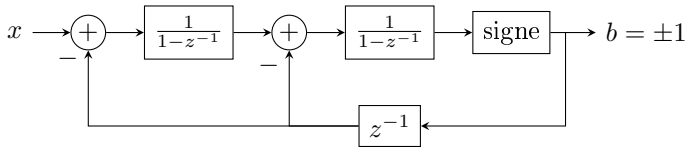
3 Partie 2 – CAN sigma-delta 1 bit

Suréchantillonner pour gagner des bits. Le bruit de quantification se répartit sur $[0, F_s'/2]$. Échantillonner à $F_s' = \text{OSR} \times F_s$ puis ne garder que $[0, F_s/2]$ divise le bruit par OSR : +3 dB par doublement. Combiné à une mise en forme d'ordre L, le gain devient $(2L+1) \times 3$ dB par doublement :

$$\text{SNR} \approx 6{,}02\,B + 1{,}76 - 10\log\frac{\pi^{2L}}{2L+1} + (2L+1)\,10\log(\text{OSR}) \quad (+\,\text{niveau}_{\text{dBFS}}).$$

Avec $B = 1$ (un simple comparateur !), $L = 2$ et OSR $= 64$, on vise ≈ 80 dB.

Le modulateur. Il compare un signal intégré à zéro, et retranche à l'entrée la décision $b = \pm 1$ (en pratique $\pm 32\,768$, pleine échelle Q15) :



ordre 1 : $i_1 \mathrel{+}= x - b,\ b = \text{signe}(i_1)$ ordre 2 : $i_1 \mathrel{+}= x - b,\ i_2 \mathrel{+}= i_1 - b,\ b = \text{signe}(i_2)$

On montre que $B(z) = X(z) + (1 - z^{-1})^L E(z)$: on retrouve la mise en forme de la partie 1, mais à la cadence OSR$\times F_s$.

Le filtre de décimation CIC. Il faut maintenant filtrer le flux de bits et revenir à F_s. Le filtre CIC (*Cascaded Integrator-Comb*, Hogenauer 1981) réalise $H(z) = \left[(1 - z^{-R})/(1 - z^{-1})\right]^K$ (moyenne glissante sur $R = \text{OSR}$ échantillons, répétée K fois) **sans aucune multiplication** : K intégrateurs à OSR$\times F_s$, on garde une valeur sur R, puis K dérivateurs (« peignes ») $d[n] = v[n] - v[n-1]$ à F_s. Règle : $K = L + 1$. Le gain continu vaut R^K.

> ⚠ **Attention**
>
> Les intégrateurs du CIC **débordent** : c'est normal ! On calcule en `uint32_t` (arithmétique modulo 2^{32}) ; tant que le résultat final tient sur 32 bits ($64^3 = 262\,144$), les débordements des intégrateurs sont exactement compensés par les soustractions des peignes.

Activité 5

Q8. Dans `effect_sigmadelta_adc.cpp`, écrire le modulateur d'ordre 1 (calcul en entiers).
Q9. Écrire les K peignes. Tester (CC 27 = ordre 1) : à l'oscilloscope ou au casque, le signal doit être reconnaissable.
Q10. Ajouter le modulateur d'ordre 2.
Q11. Sinus à -6 dBFS, 1 kHz, $B = 16$: remplir le tableau (SINAD et ENOB) et comparer à la formule. Combien de dB gagne-t-on par doublement d'OSR ?

Q12. Ordre 2, OSR 64 : augmenter le niveau jusqu'à 0 dBFS. Que se passe-t-il ? Puis passer le sinus à 5 kHz : que devient le niveau mesuré ? Pourquoi ?

	OSR 16		OSR 32		OSR 64	
	SINAD	ENOB	SINAD	ENOB	SINAD	ENOB
ordre 1						
ordre 2						

```
for (int r = 0; r < R; r++) {
  // ----- Modulateur ΣΔ (b = ±32768 : la pleine échelle) -----
  // TODO (Q8)  : ordre 1 : i1 += x - b ;  b = +32768 si i1 >= 0, sinon -32768
  // TODO (Q10) : ordre 2 : i1 += x - b ;  i2 += i1 - b ;  b = signe(i2)·32768
  b = (x >= 0) ? 32768 : -32768;      // provisoire : simple comparateur 1 bit

  const int32_t bit = (b > 0) ? 1 : -1;
  nUns += (bit > 0);

  // ----- Intégrateurs du CIC, à OSR·Fs -----
  integ_[0] += (uint32_t)bit;
  for (int k = 1; k < K; k++) integ_[k] += integ_[k - 1];
}

// ----- Décimation : 1 valeur sur R, puis peignes à Fs -----
uint32_t v = integ_[K - 1];
// TODO (Q9) : K peignes en cascade, pour k = 0..K-1 :
//             d = v - peigne_[k] ;  peigne_[k] = v ;  v = d
float y = (float)(int32_t)v * gain;          // ramené dans [-1, 1]
```

® **Remarque** : Pourquoi des entiers ?

Le modulateur est codé en `int32_t` : le calcul est exact, comme dans un circuit numérique, et rapide sur le Cortex-M7. Une version en `float` fonctionne à OSR 64, mais en simulation à OSR 256 (partie 3) elle plafonne à ≈ 68 dB contre ≈ 89 dB en entiers : les arrondis de l'intégrateur finissent par se voir.

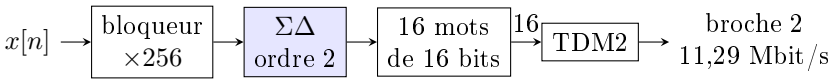
4 Partie 3 – CNA 1 bit sur une broche (GPIO)

⚠ **Attention**

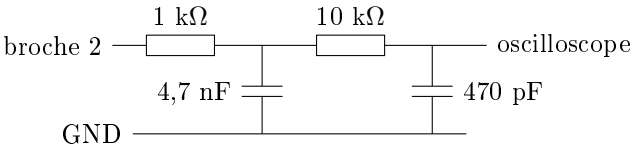
Partie **expérimentale** : le principe s'appuie sur le code source de la bibliothèque Audio (`AudioOutputTDM2`) ; le flux a été validé en simulation (décodage des trames), le comportement électrique est **à valider sur carte**.

L'astuce. Un modulateur ΣΔ produit un bit par période d'horloge rapide. Il faut donc une broche capable de sortir des bits *réguliers* à plusieurs MHz, sans gigue : un `digitalWrite()` dans une boucle en est incapable. Le second port audio série de la Teensy 4 (SAI2), piloté par `AudioOutputTDM2`, l'est : à chaque période $1/F_s$, il émet sur la **broche 2** une trame de 8 mots de 32 bits, soit **256 bits sans aucun trou**, à BCLK $= 256\,F_s = 11{,}29$ MHz (broche 4). Ses 16 entrées de 16 bits sont émises dans

l'ordre (entrée 0 d'abord), bit de poids fort en premier. Il suffit d'y ranger les 256 bits successifs d'un modulateur tournant à $256\,F_s$: la broche 2 devient la sortie d'un CNA sigma-delta d'OSR 256.



Câblage. Filtre passe-bas RC d'ordre 2 (seconde cellule 10 fois plus impédante pour ne pas charger la première) :



Activité 6

Q13. Dans `bitstream_gpio.cpp`, écrire le modulateur (le même qu'en partie 2) et l'empilement des bits dans `mot`, de sorte que le premier bit calculé soit le bit de poids fort. Pourquoi cet ordre ?

Q14. Calculer la fréquence de coupure de chaque cellule RC. Quelle atténuation à 11,29 MHz ? Quelle tension continue et quelle amplitude attend-on en sortie pour un sinus à -6 dBFS ?

Q15. Activer le flux (CC 83). À l'oscilloscope : observer la broche 2 (base de temps de l'ordre de la µs), puis la sortie du filtre. Faire varier le niveau et la fréquence. Avec la FFT de l'oscilloscope, repérer la raie du signal, le bruit rejeté vers les hautes fréquences, et les raies images autour de F_s.

Q16. Quel SNR la théorie prévoit-elle à OSR 256, ordre 2 ? Citer trois raisons pour lesquelles la sortie analogique réelle sera bien moins bonne.

```
for (int c = 0; c < BS_CANAUX; c++) {
  uint32_t mot = 0;
  for (int k = 0; k < 16; k++) {
    // ----- Modulateur ΣΔ à 256·Fs, en entiers (calcul exact) -----
    // TODO (Q13) : même modulateur que dans le CAN (ordre 1 ou 2)
    b = (x >= 0) ? 32768 : -32768;    // provisoire

    // TODO (Q13) : empiler le bit (1 si b > 0) dans 'mot' de sorte que
    //              le 1er bit calculé soit le bit de poids fort (émis en 1er)
  }
  out[c]->data[i] = (int16_t)(uint16_t)mot;
}
```

5 Livrables

1. Requantificateur avec dither et mise en forme ; tableau de Q7 commenté ;
2. Tableau de Q11 et comparaison à la formule $(3(2L+1)$ dB par doublement d'OSR) ;
3. Relevés oscilloscope de la partie 3 (broche 2, sortie du filtre, FFT).

® **Remarque** : Pour aller plus loin

Boucler la sortie RC sur l'entrée **LINE IN** de l'Audio Shield (via un condensateur de liaison) et la mesurer avec le même banc (`AudioInputI2S` vers la file de mesure) ; placer le requantificateur de la partie 1 *avant* le CNA 1 bit ; étudier les modulateurs d'ordre supérieur (stabilité, MASH).